A REFINEMENT BOUNDARY FOR REVERSIBLE NVIDIA EXECUTION

BY ANONYMOUS AUTHOR

Roop emits PTX and can execute generated kernels on NVIDIA hardware. A passing hardware test establishes a useful regression fact, but it does not prove that a GPU execution implements the Lean source semantics. We formulate the source, compiler, and backend relations needed for such a result. A conditional composition theorem shows exactly how the relations imply executable reversibility. The current repository establishes a source-level certificate calculus for a restricted class of transformations and records a PTX launch and copyback test on an RTX 4090. The compiler correspondence and PTX refinement hypotheses remain open.

1. The question. PTX is a virtual instruction set used in CUDA compilation [2]. A source-level Lean theorem and a successful PTX launch answer different questions. The former concerns a mathematical transition system. The latter checks one concrete executable and environment. This paper states the relation required to carry a reversible source theorem to a running GPU program.

Let S be a source state, M a device-memory state, and P a well-formed source program. Write $\mathrm{Sem}(P) : S \to S$ for the source transition of a terminating reversible fragment. Let $\mathrm{inv}(P)$ be its generated inverse, $K(P)$ its emitted kernel, and $\mathrm{run}(K(P), m)$ the observed device transition.

DEFINITION 1. A relation $R \subseteq M \times S$ is a representation relation when $R(m, s)$ states that device memory m represents source state s. A kernel K refines a source transition F through R, written $K \sqsubseteq_R F$, when

$$R(m, s) \implies R(\mathrm{run}(K, m), F(s)).$$

The definition hides the details of allocation, transfers, thread geometry, and synchronization in the device transition and in R. A useful proof must state those details for its chosen kernel class.

2. A conditional end-to-end theorem.

THEOREM 2 (executable round trip). *Assume the following for a program P.*

1. *Source reversal:* $\mathrm{Sem}(\mathrm{inv}(P))(\mathrm{Sem}(P)(s)) = s$ *for every source state* s.
2. *Forward refinement:* $K(P) \sqsubseteq_R \mathrm{Sem}(P)$.
3. *Inverse refinement:* $K(\mathrm{inv}(P)) \sqsubseteq_R \mathrm{Sem}(\mathrm{inv}(P))$.
4. *Observability:* $R(m, s)$ *and* $R(m', s')$ *determine equality of the host-visible values under comparison.*

Then, for every m, s with $R(m, s)$, executing $K(P)$ followed by $K(\mathrm{inv}(P))$ produces a device state representing s. The host-visible state equals its initial observed value.

MSC2020 subject classifications. Primary 68N20.
Keywords and phrases. CUDA, compiler verification, reversible computation.

PROOF. Forward refinement yields

$$R(\mathrm{run}(K(P), m), \mathrm{Sem}(P)(s)).$$

Apply inverse refinement to obtain a state m' satisfying

$$R(m', \mathrm{Sem}(\mathrm{inv}(P))(\mathrm{Sem}(P)(s))).$$

Source reversal reduces the right side to s. Observability gives equality of the compared host-visible values. □

The theorem is deliberately conditional. A source theorem cannot establish either refinement statement by itself. Likewise, a regression test cannot quantify over all represented states.

3. Current evidence. The repository records passing CUDA runtime tests on an NVIDIA GeForce RTX 4090 through driver 595.99.02. One test launches PTX and checks writable-buffer copyback. Another launches a hand-written two-buffer kernel that computes $p_i \leftarrow (p_i + q_i) - q_i$ at four coordinates, including `i64::MIN` and `i64::MAX`, and checks exact restoration. An end-to-end regression checks the Lean model for the same four-coordinate Roop source, compiles it to PTX, runs it and its generated inverse, and checks exact restoration. These are paired checks for one program, not a compiler refinement proof. They do not establish a representation relation, absence of races, numerical equivalence, or behavior for arbitrary kernel launches.

The source side is stronger for the additive coupling calculus. The checked Lean module proves exact modular round trips for fixed-width tensors and arbitrary finite lists of certified transformations. The `roop weave -certificate` command emits a checked plan with opaque update symbols and an FNV-1a fingerprint of the emitted Roop source. The theorem in this paper would apply to that fragment once the two refinement hypotheses are proved.

4. A first refinement target. A narrow first target is the integer update

$$p_i \leftarrow p_i + f(q)_i \pmod{2^{64}},$$

where the kernel does not write q. Its source inverse is

$$p_i \leftarrow p_i - f(q)_i \pmod{2^{64}}.$$

The required representation relation maps each source tensor coordinate to the corresponding device buffer word. The compiler theorem must show that the emitted index expression covers each intended coordinate once and that the alias conditions keep the read-only q buffer distinct from the updated p buffer. The backend theorem must then cover the declared grid, block size, memory accesses, and synchronization policy.

This target does not prove a general CUDA memory model theorem. It produces one auditable refinement result whose source contract already has a Lean certificate. Further kernels can reuse its memory representation and proof structure only after their own indexing and alias obligations are discharged.

5. Development status. The present development has four distinct claims.

1. The generic source certificate calculus is checked in Lean.
2. Generated PTX launches and copies writable buffers in a recorded RTX 4090 test.
3. No theorem currently relates emitted Roop GPU code to the source certificate calculus.
4. No benchmark in the repository establishes CUDA throughput or competitiveness.

The next implementation milestone is therefore not a larger performance claim. It is a generated, machine-checkable contract for one additive kernel together with a proof that the compiler chooses that kernel only when its contract applies.

REFERENCES

[1] L. de Moura and S. Ullrich (2021). The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, 625–635.

[2] NVIDIA Corporation (2025). *CUDA C++ Programming Guide*, Release 13.2.